

计算机图形学原理

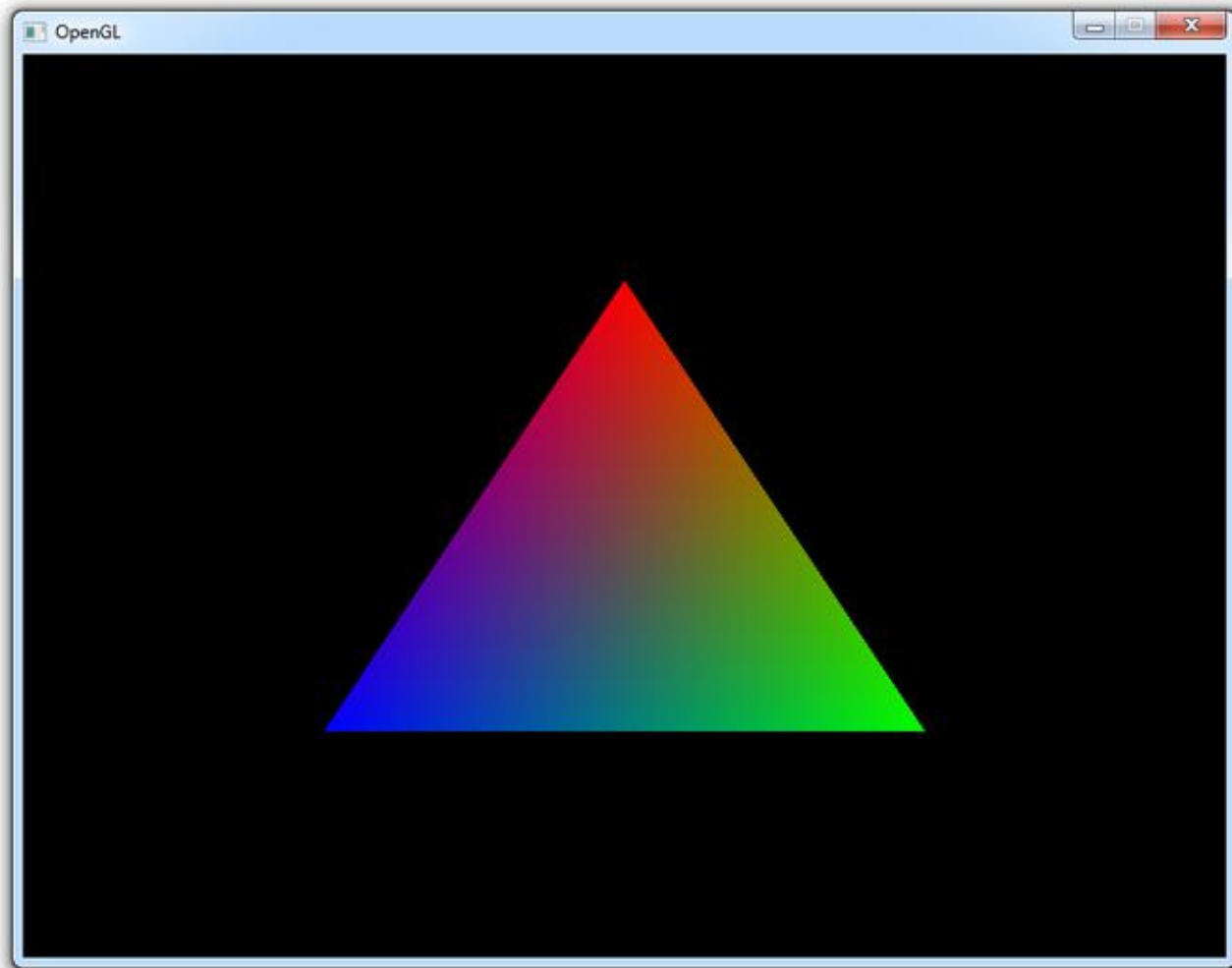
-----教程5

李瑞辉

很好的自学参考:

<https://learnopengl.com/> (英文)

<https://learnopengl-cn.github.io/> (中文)



逼真的灯光



秘诀在于照明

大纲

➤ 法向量

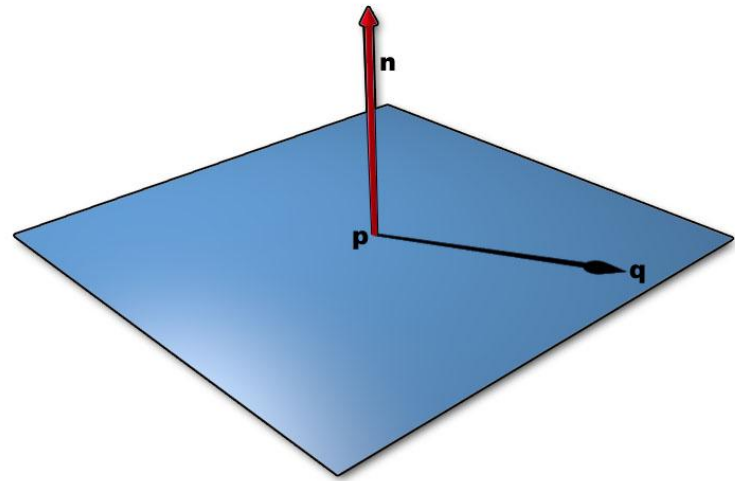


➤ OpenGL光照系统

法向量

➤ 平面的法向量

- 长度 = 1
- 垂直于平面



法向量

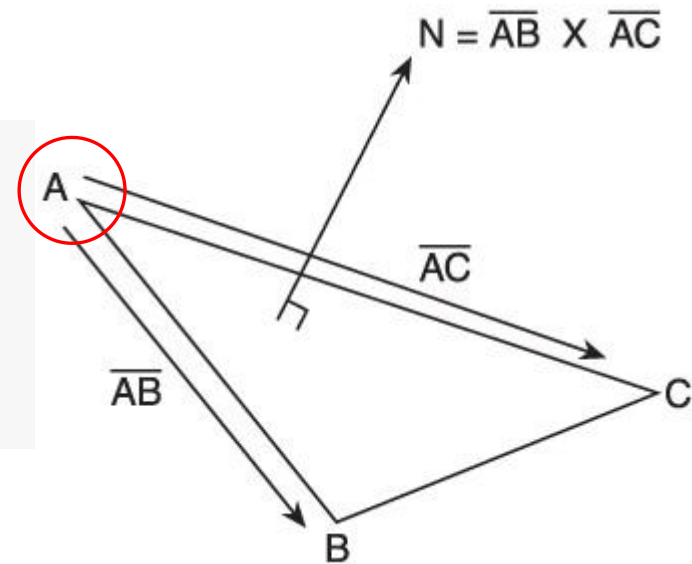
➤三角形的法向量（模型的面）

- 长度 = 1
- 垂直于三角形（逆时针或顺时针）

三角形中两个向量的叉积

伪代码

```
triangle ( v1, v2, v3 )  
edge1 = v2-v1  
edge2 = v3-v1  
triangle.normal = cross(edge1, edge2).normalize()
```

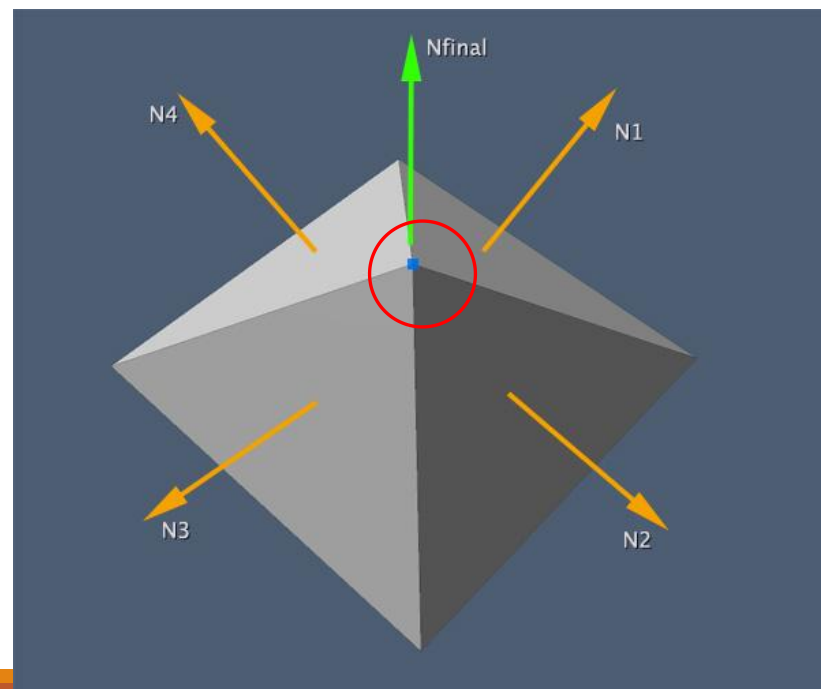


法向量

➤ 顶点法向量

- 长度 = 1
- 包含顶点的三角形曲面的法线平均值
- 顶点的一个属性，

喜欢位置和颜色



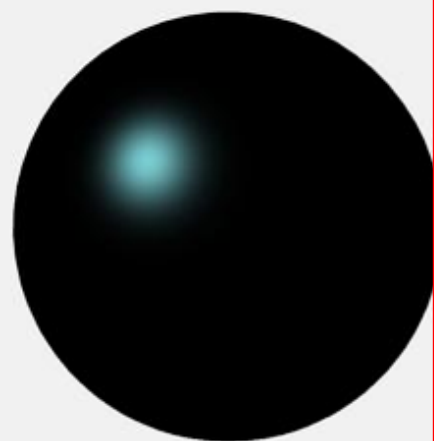
灯光效果



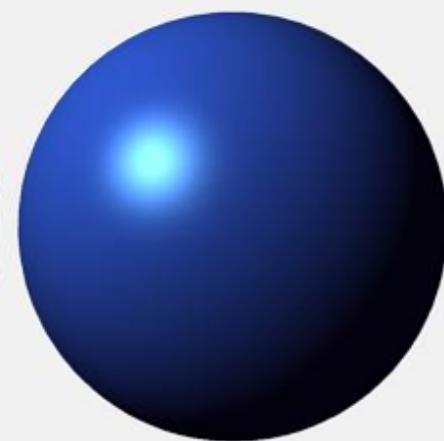
Ambient



Diffuse



Specular



Combined



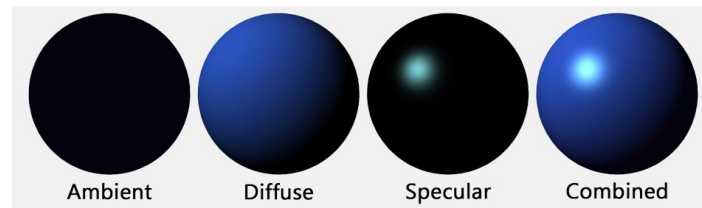
OpenGL光照

- 光源的颜色由它们**发出的**
- 在各个方向**反射的**入射 R、G、B 分量的百分比
- 场景中的光来自多个可以单独打开和关闭的光源
- **中**，光照分为四个独立的部分：**E**missive、**A**mbient、**D**iffuse和**S**pecular
- 所有四个组件都是独立计算的，然后加在一起
颜色 = $E + A + D + S$

法向量

➤ 为什么对照明很重要

- 冲击亮度漫射光 ←
- 决定反射方向镜面光 ←



➤ 如何访问每个顶点的法向量

- 法线
- 加载。 **对象** 已有此信息的文件

环境光

- 被环境散射太多以至于无法确定其方向的光（例如月亮，远处的光）
- 似乎来自四面八方

房间内的背光



- 大环境分量
- 光线经过许多表面反射后到达您

户外聚光灯



- 微小的环境组件
- 大部分光沿同一方向传播
- 其他物体反射后，只有很少一部分光到达您的眼睛

环境光窗口

- 在main.cpp中

```
-void MeGlWindow::paintGL()
{
    glClear(GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT);
    glViewport(0, 0, width(), height());

    mat4 fullTransformMatrix;
    mat4 viewToProjectionMatrix = glm::perspective(60.0f, ((float)width()) / height(), 0.1f, 20.0f);
    mat4 worldToViewMatrix = camera.getWorldToViewMatrix();
    mat4 worldToProjectionMatrix = viewToProjectionMatrix * worldToViewMatrix;

    GLint ambientLightUniformLocation = glGetUniformLocation(programID, "ambientLight");
    vec3 ambientLight(0.1f, 0.1f, 0.1f);
    glUniform3fv(ambientLightUniformLocation, 1, &ambientLight[0]);
}
```

RGB值

环境光着色器

- 在顶点着色器中

```
#version 430

in layout(location=0) vec3 position;
in layout(location=1) vec3 vertexColor;
uniform vec3 ambientLight;

uniform mat4 fullTransformMatrix;

out vec3 theColor;

void main()
{
    vec4 v = vec4(position, 1.0);
    gl_Position = fullTransformMatrix * v;
    theColor = vertexColor * ambientLight;
}
```

环境光着色器

- 在顶点着色器中

```
#version 430

in layout(location=0) vec3 position;
in layout(location=1) vec3 vertexColor;
uniform vec3 ambientLight;

uniform mat4 fullTransformMatrix;

out vec3 theColor;

void main()
{
    vec4 v = vec4(position, 1.0);
    gl_Position = fullTransformMatrix * v;
    theColor = vertexColor * ambientLight;
}
```

两个向量积

- 不是点（标量）
- 不交叉
- 简单组件产品

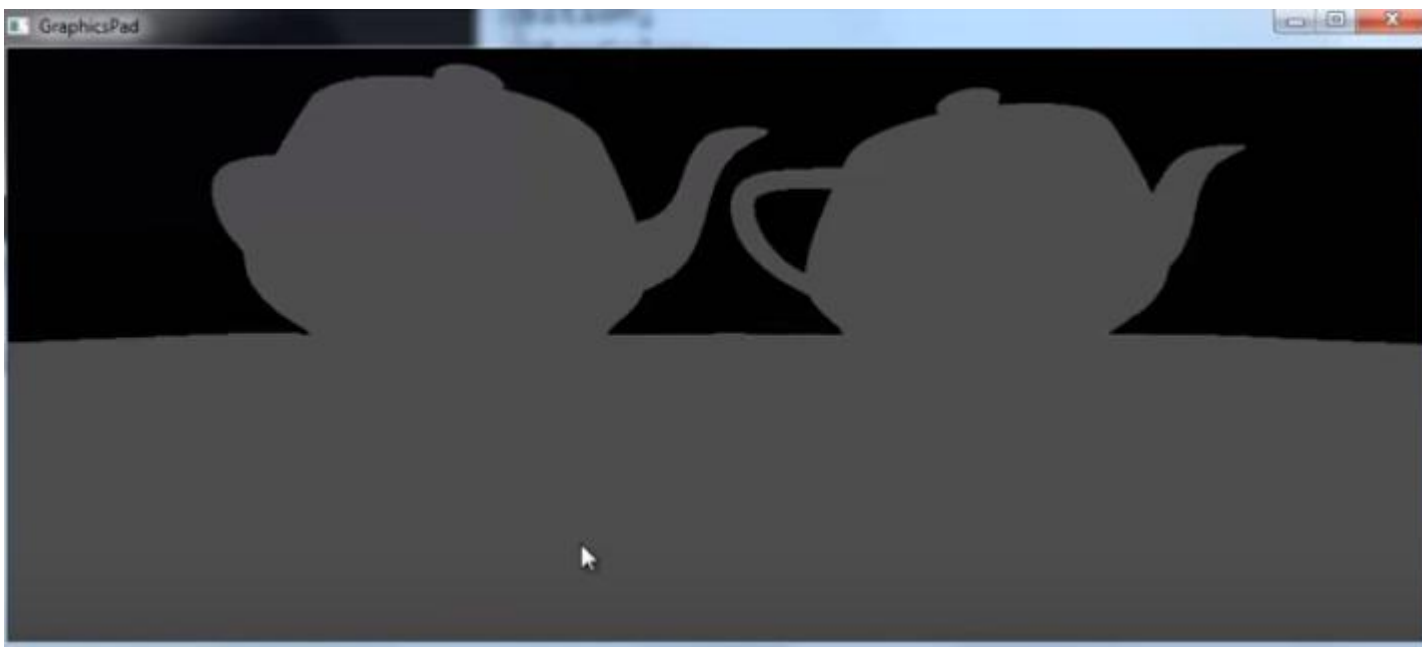
顶点颜色： $\langle 1, 0.5, 0.1 \rangle$

环境光： $\langle 0.5, 0.5, 0.9 \rangle$

结果： $\langle 0.5, 0.25, 0.09 \rangle$

环境光着色器

- 在顶点着色器中

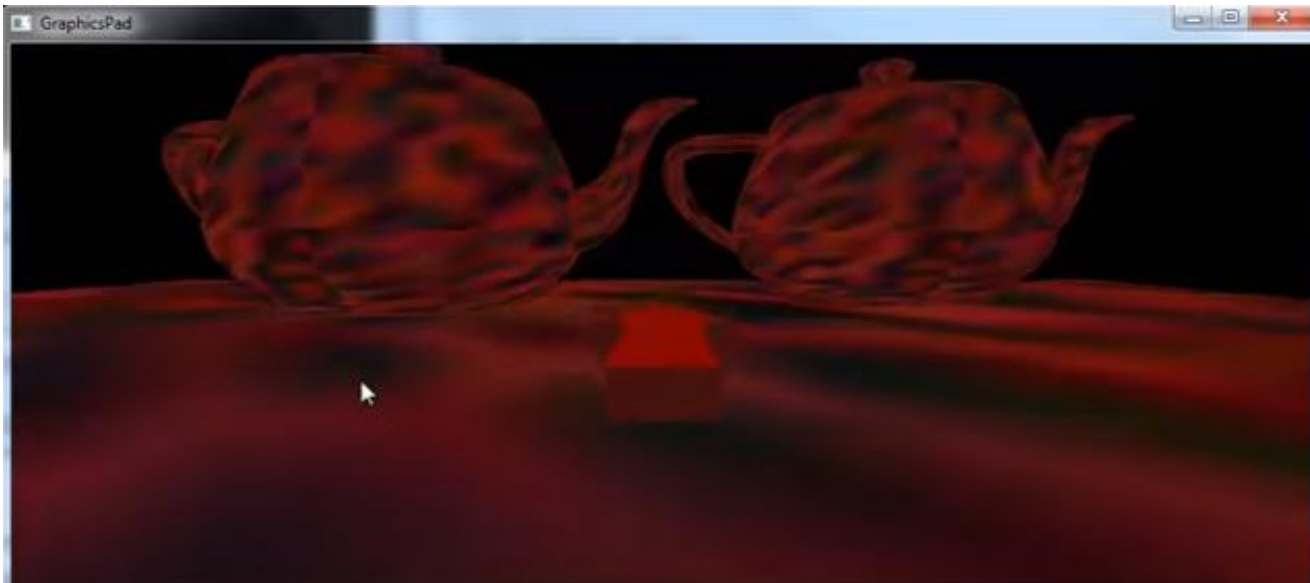


vec3环境光(0.5f, 0.5f, 0.5f)

红绿蓝

环境光窗口

- 在main.cpp中

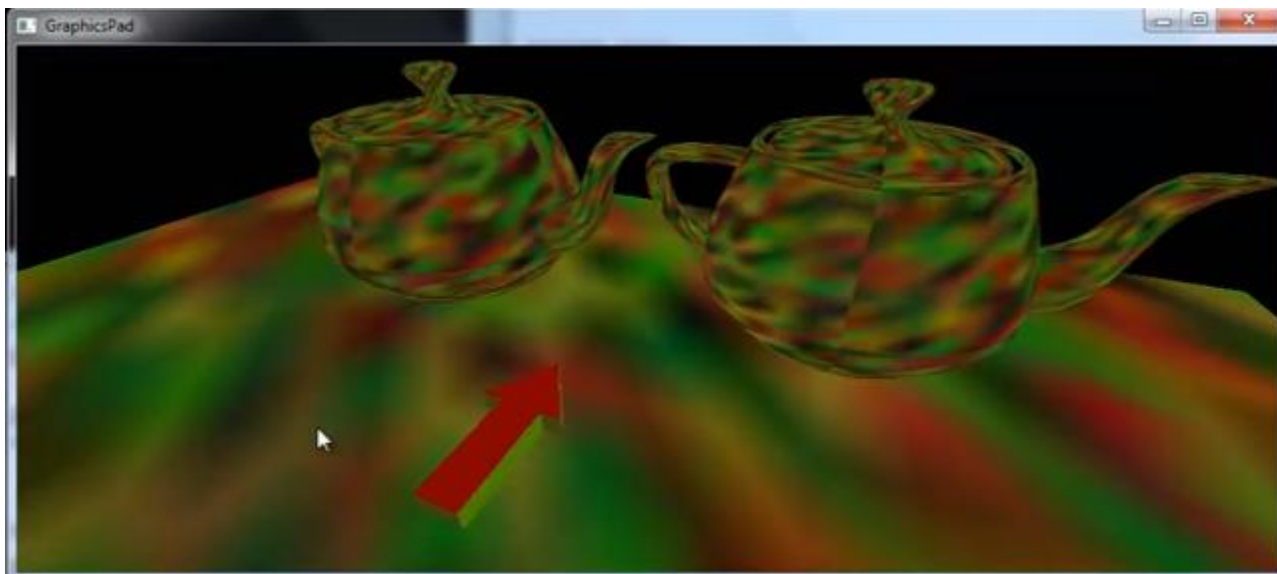


vec3环境光(0.5f, 0.1f, 0.1f)

红绿蓝

环境光窗口

- 在main.cpp中

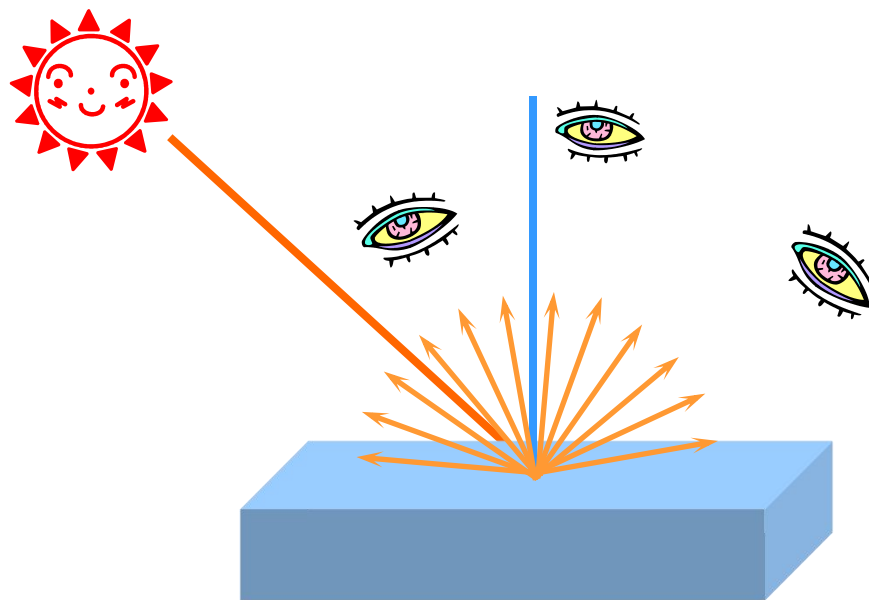


vec3环境光(0.5f, 0.5f, 0.1f)

红绿蓝

漫射光

- 来自一个方向
- 一旦它碰到一个表面，它就会均匀地散布在各个方向
- 无论眼睛位于何处，物体都显得同样明亮

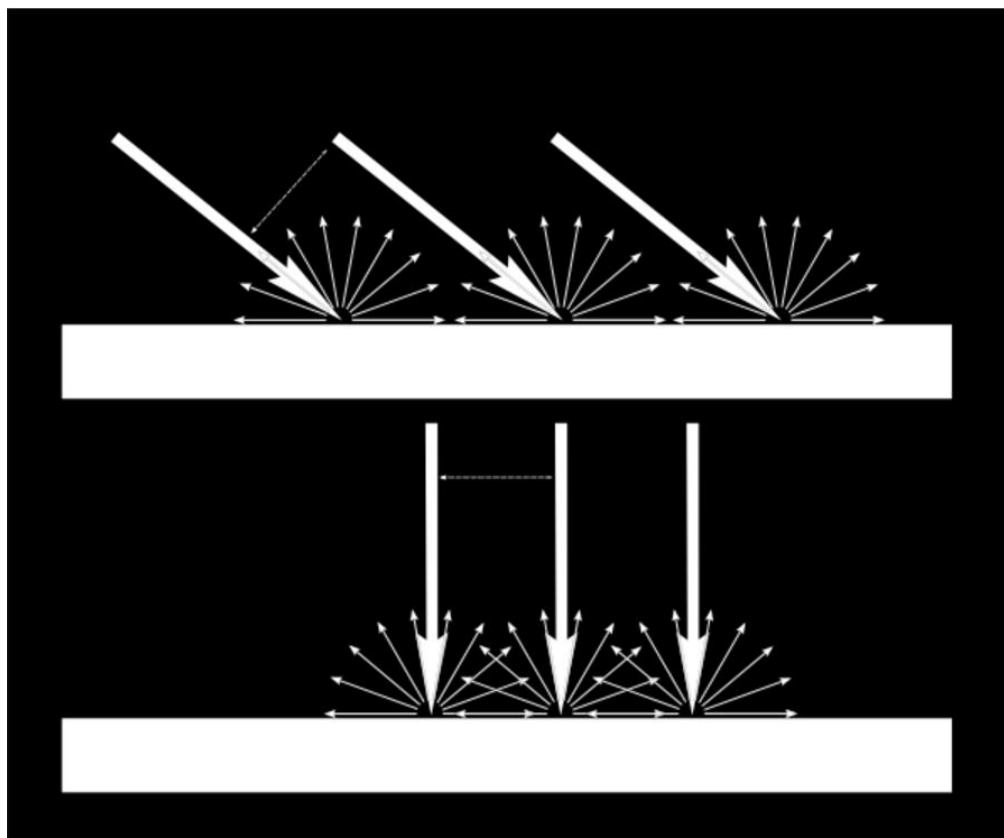


漫射光

- 亮度由光照方向与法线的夹角决定

更小

更亮



漫射光

- 在 main.cpp 中

- 法向量
- 灯位

```
//vertex position
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), 0);
//vertex color
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (char*)(3 * sizeof(float)));
//vertex normal
glEnableVertexAttribArray(2);
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (char*)(6 * sizeof(float)));
```

发送正常信息，如位置和颜色

```
GLint lightPositionUniformLocation =
    glGetUniformLocation(programID, "lightPositionWorld");
vec3 lightPosition(2.0f, 15.0f, -10.0f);
glUniform3fv(lightPositionUniformLocation, 1, &lightPosition[0]);
```

设置光源位置
[同样可以设置颜色]

漫反射光着色器

- 在VertexShader中： (1)从 OpenGL 获取法线信息

```
//vertex position
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), 0);
//vertex color
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (char*)(3 * sizeof(float)));
//vertex normal
glEnableVertexAttribArray(2);
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (char*)(6 * sizeof(float)));
```



```
#version 430

in layout(location=0) vec3 position;
in layout(location=1) vec3 vertexColor;
in layout(location=2) vec3 normal;

uniform mat4 modelTransformMatrix;
uniform mat4 projectionMatrix;

out vec3 theColor;
out vec3 normalWorld;
out vec3 vertexPositionWorld;
```

漫反射光着色器

- 在顶点着色器中:

```
#version 430
```

```
in layout(location=0) vec3 position;  
in layout(location=1) vec3 vertexColor;  
in layout(location=2) vec3 normal;
```

```
uniform mat4 modelTransformMatrix;  
uniform mat4 projectionMatrix;
```

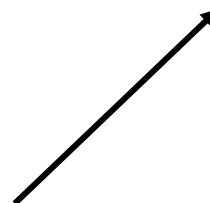
```
out vec3 theColor;  
out vec3 normalWorld;  
out vec3 vertexPositionWorld;
```

```
void main()
```

```
{  
    vec4 v = vec4(position, 1.0);  
    vec4 newPosition = modelTransformMatrix * v;  
    vec4 projectedPosition = projectionMatrix * newPosition;  
    gl_Position = projectedPosition;  
  
    vec4 normal_temp = modelTransformMatrix * vec4(normal, 0);  
    normalWorld = normal_temp.xyz;  
  
    vertexPositionWorld = newPosition.xyz;  
  
    //theColor = vertexColor;  
}
```

(2) 计算顶点和法线的世界坐标
即模型矩阵*模型坐标

法向量的计算



漫反射光着色器

- 在片段着色器中:
- (1)从VertexShader

```
#version 430

in layout(location=0) vec3 position;
in layout(location=1) vec3 vertexColor;
in layout(location=2) vec3 normal;

uniform mat4 modelTransformMatrix;
uniform mat4 projectionMatrix;

out vec3 theColor;
out vec3 normalWorld;
out vec3 vertexPositionWorld;
```

顶点着色器



```
#version 430

out vec4 daColor;
in vec3 normalWorld;
in vec3 vertexPositionWorld;

uniform vec4 ambientLight;
uniform vec3 lightPositionWorld;
uniform vec3 eyePositionWorld;
```

片段着色器

漫反射光着色器

- 在片段着色器中:
- (2)从 OpenGL

```
GLint lightPositionUniformLocation =  
    glGetUniformLocation(programID, "lightPositionWorld");  
vec3 lightPosition(2.0f, 15.0f, -10.0f);  
glUniform3fv(lightPositionUniformLocation, 1, &lightPosition[0]);
```

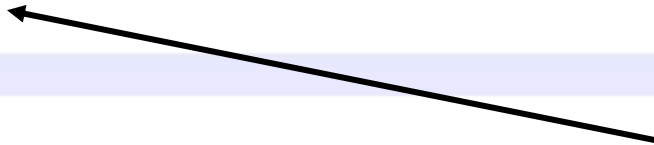


```
#version 430  
  
out vec4 daColor;  
in vec3 normalWorld;  
in vec3 vertexPositionWorld;  
  
uniform vec4 ambientLight;  
uniform vec3 lightPositionWorld;  
uniform vec3 eyePositionWorld;
```

片段着色器

漫反射光着色器

- 在片段着色器中:
- (3) 计算光线方向
- (4) 通过点积计算亮度



```
void main()  
{  
    // Diffuse  
    vec3 lightVectorWorld = normalize(lightPositionWorld - vertexPositionWorld);  
    float brightness = dot(lightVectorWorld, normalize(normalWorld));  
    vec4 diffuseLight = vec4(brightness, brightness, brightness, 1.0);  
}
```

- (5) 输出最终颜色

```
daColor = ambientLight + clamp(diffuseLight, 0, 1)
```

漫反射光着色器

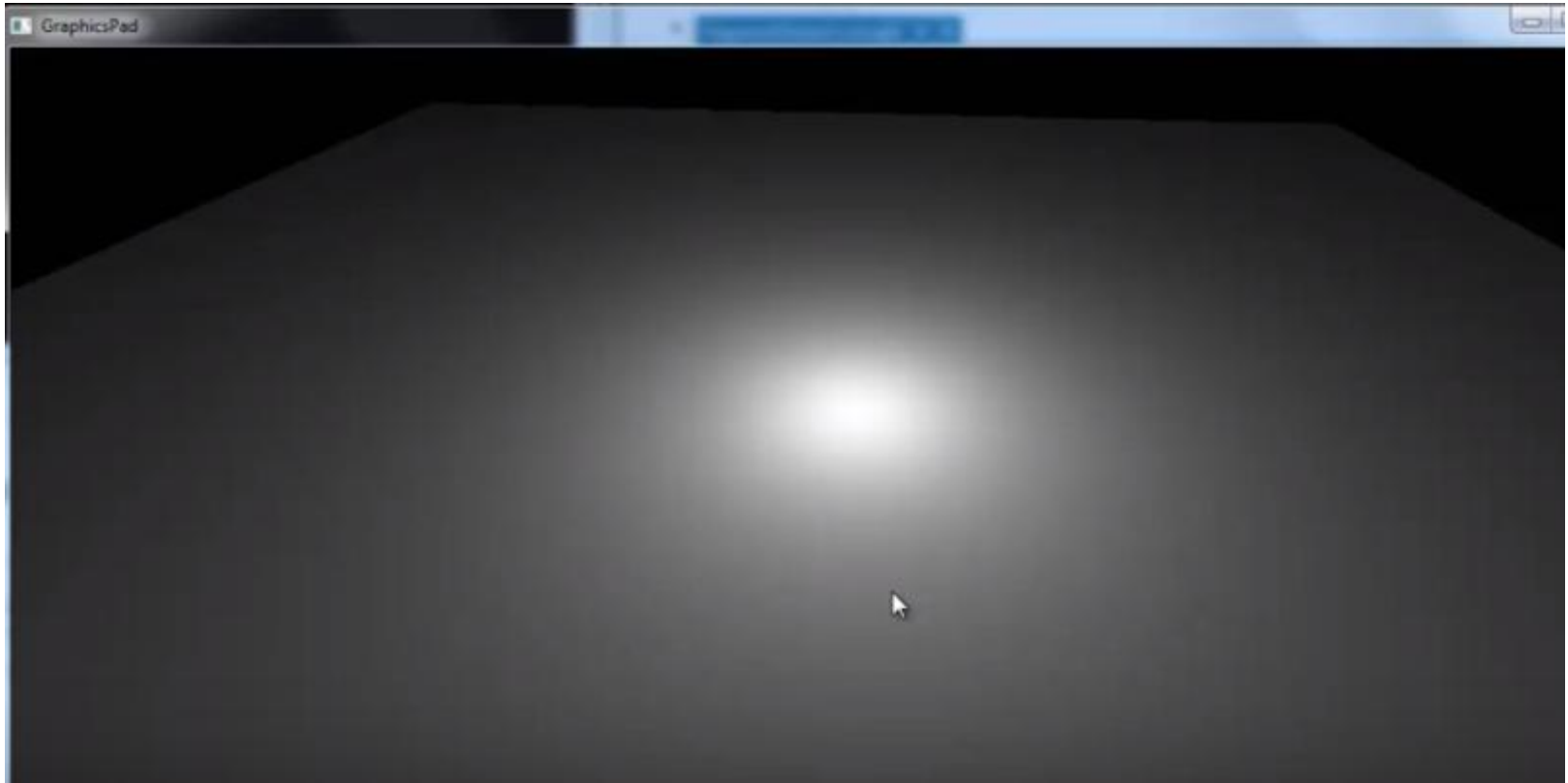
- 在片段着色器中：

```
void main()  
{  
    // Diffuse  
    vec3 lightVectorWorld = normalize(lightPositionWorld - vertexPositionWorld);  
    float brightness = dot(lightVectorWorld, normalize(normalWorld));  
    vec4 diffuseLight = vec4(brightness, brightness, brightness, 1.0);  
  
    daColor = ambientLight + clamp(diffuseLight, 0, 1)
```

- 点积的结果可能为负
- 使用clamp让值在0到1之间

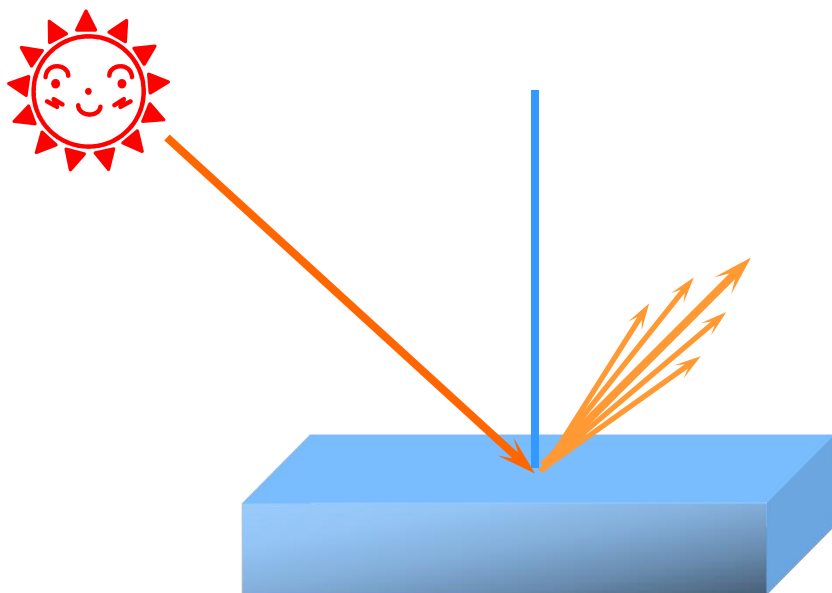


漫射光效果



镜面光

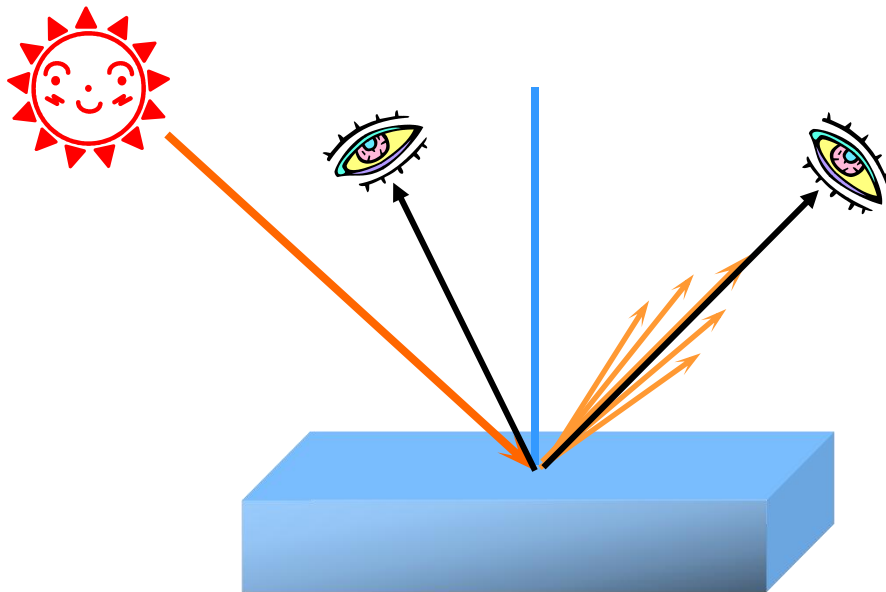
- 来自特定方向
- 倾向于以首选方向从表面反弹
- 将镜面反射视为光泽



例如，从镜子反射的激光束产生
几乎 100% 的镜面反射

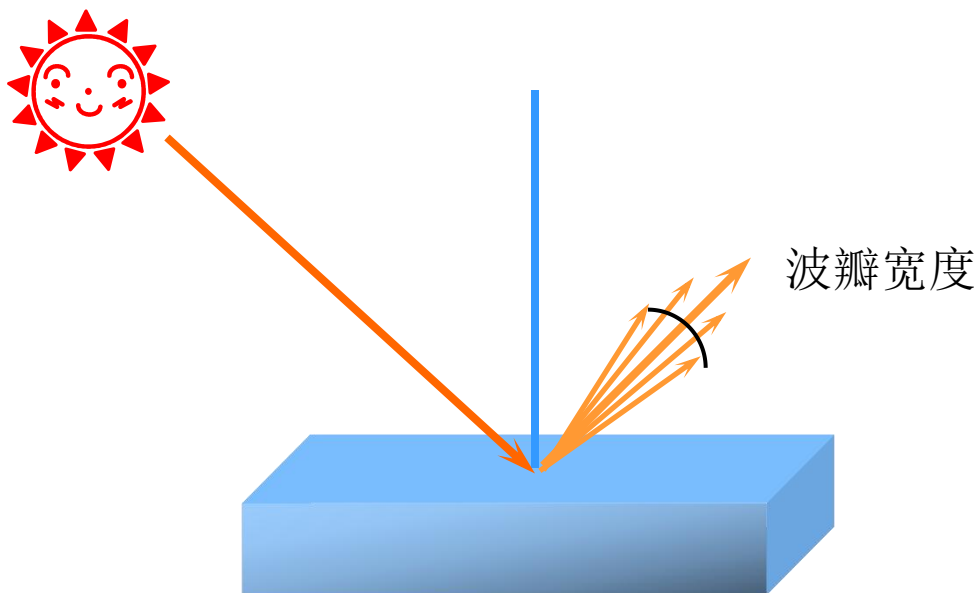
镜面光

- 亮度由眼睛之间的角度和反射光方向决定



镜面光

- 波瓣宽度由模型材料决定
- 当为镜面时，波瓣宽度为0，即所有反射光的方向相同。



镜面光-主

- 在main.cpp中

```
GLint eyePositionUniformLocation =  
    glGetUniformLocation(programID, "eyePositionWorld");  
vec3 eyePosition(0.0f, 0.0f, 0.0f);  
glUniform3fv(eyePositionUniformLocation, 1, &eyePosition[0]);  
  
GLint lightPositionUniformLocation =  
    glGetUniformLocation(programID, "lightPositionWorld");  
vec3 lightPosition(2.0f, 15.0f, -10.0f);  
glUniform3fv(lightPositionUniformLocation, 1, &lightPosition[0]);
```

除了正常信息和光源位置外，还向 OpenGL 发送眼睛位置

镜面光 -着色器

- fragmentShader中： (1) 从OpenGL获取眼睛位置信息

```
GLint eyePositionUniformLocation =  
    glGetUniformLocation(programID, "eyePositionWorld");  
vec3 eyePosition(0.0f, 0.0f, 0.0f);  
glUniform3fv(eyePositionUniformLocation, 1, &eyePosition[0]);
```

```
GLint lightPositionUniformLocation =  
    glGetUniformLocation(programID, "lightPositionWorld");  
vec3 lightPosition(2.0f, 15.0f, -10.0f);  
glUniform3fv(lightPositionUniformLocation, 1, &lightPosition[0]);
```



```
#version 430
```

```
out vec4 daColor;  
in vec3 normalWorld;  
in vec3 vertexPositionWorld;
```

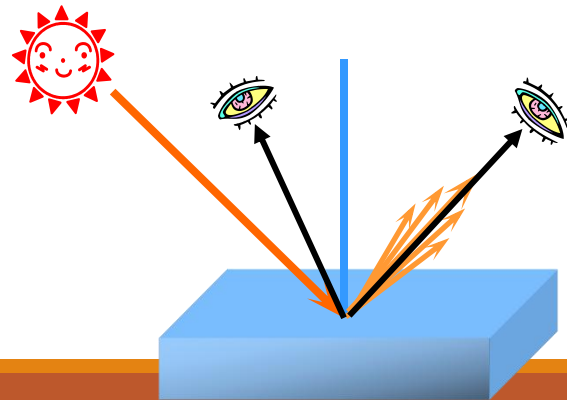
```
uniform vec4 ambientLight;a  
uniform vec3 lightPositionWorld;  
uniform vec3 eyePositionWorld;
```

镜面光 - 着色器

- 在片段着色器中:

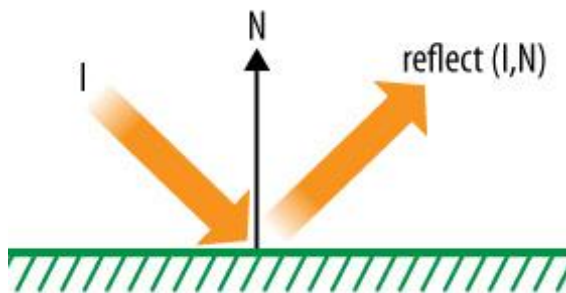
```
// Specular
//Calculate reflect light direction
vec3 reflectedLightVectorWorld = reflect(-lightVectorWorld, normalWorld);
//Calculate direction from eye to object
vec3 eyeVectorWorld = normalize(eyePositionWorld - vertexPositionWorld);
//Calculate light brightness according to the angle between eye and reflect light
float s = clamp(dot(reflectedLightVectorWorld, eyeVectorWorld), 0, 1);
//Control lobe of specularLight
s = pow(s, 50);
vec4 specularLight = vec4(s, 0, 0, 1);

daColor = ambientLight + clamp(diffuseLight, 0, 1) + specularLight;
```



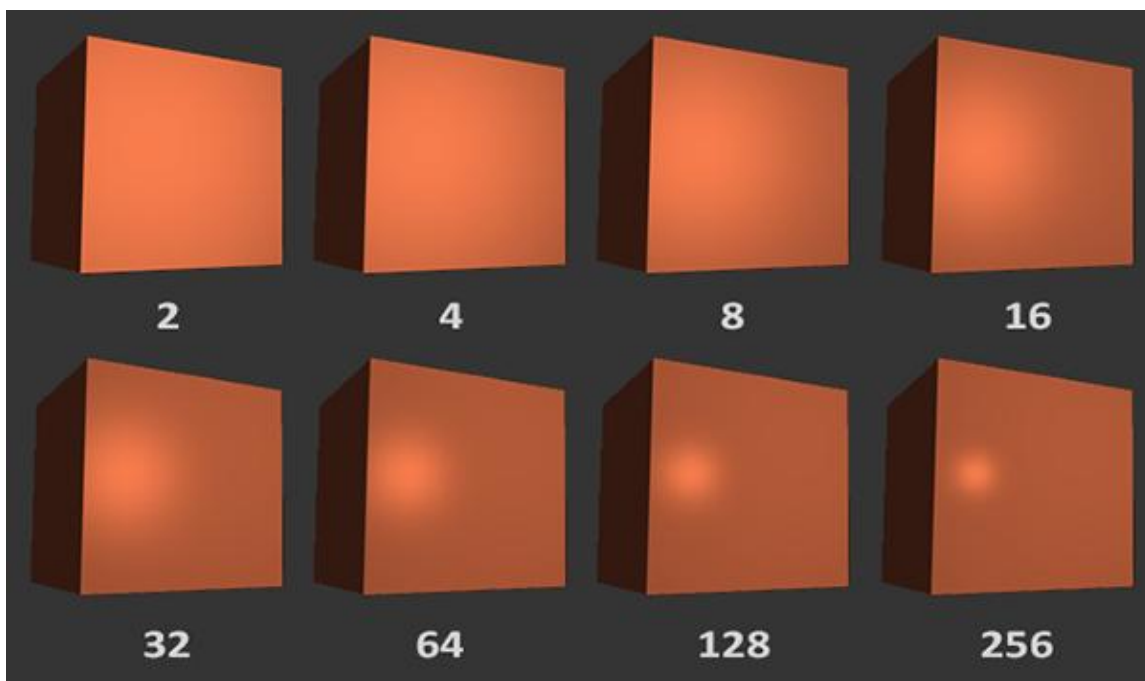
为什么 添加 消极的 符号 前 光矢量世界？

反射函数期望第一个向量从光源指向片段的位置，但光
VectorWorld向量目前指向相反的方向：从片段指向光源



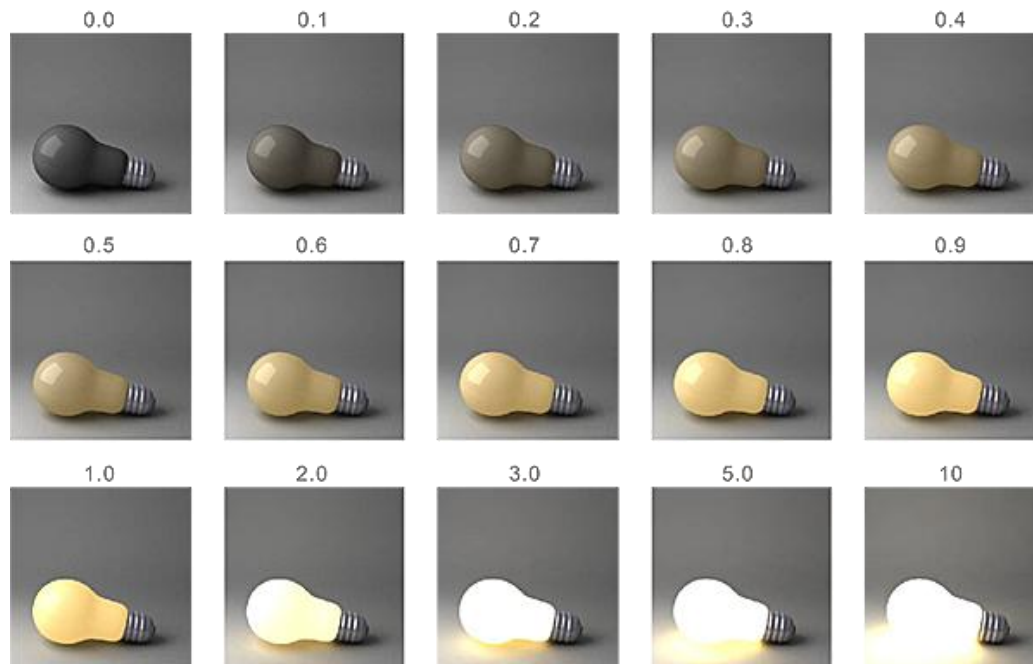
N应该归一化！

这个 50 值是高光的光泽度值。物体的光泽度值越高，它就越能正确地反射光线而不是四处散射光线，因此高光变得越小。



发射光

- 这是物体自身发出的光。一个典型的例子就是光源本身。
- 由材料特性指定



整个画面



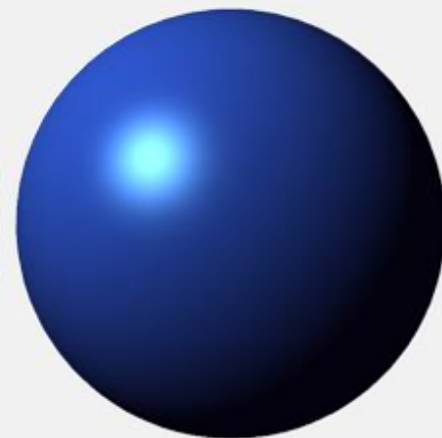
Ambient



Diffuse



Specular



Combined

下一教程

- 多个灯
- 如何加载 .obj 文件
- 如何加载纹理文件
- 纹理贴图